JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ForkJoinPool

java.lang.Object
    java.util.concurrent.AbstractExecutorService
        java.util.concurrent.ForkJoinPool

All Implemented Interfaces:

AutoCloseable, Executor, ExecutorService, ScheduledExecutorService

---

```
public class ForkJoinPool
extends AbstractExecutorService
implements ScheduledExecutorService
```

An ExecutorService for running ForkJoinTasks. A ForkJoinPool provides the entry point for submissions from non-ForkJoinTask clients, as well as management and monitoring operations.

A ForkJoinPool differs from other kinds of ExecutorService mainly by virtue of employing *work-stealing*: all threads in the pool attempt to find and execute tasks submitted to the pool and/or created by other active tasks (eventually blocking waiting for work if none exist). This enables efficient processing when most tasks spawn other subtasks (as do most ForkJoinTasks), as well as when many small tasks are submitted to the pool from external clients. Especially when setting *asyncMode* to true in constructors, ForkJoinPools may also be appropriate for use with event-style tasks that are never joined. All worker threads are initialized with Thread.isDaemon() set true.

A static commonPool() is available and appropriate for most applications. The common pool is used by any ForkJoinTask that is not explicitly submitted to a specified pool. Using the common pool normally reduces resource usage (its threads are slowly reclaimed during periods of non-use, and reinstated upon subsequent use).

For applications that require separate or custom pools, a ForkJoinPool may be constructed with a given target parallelism level; by default, equal to the number of available processors. The pool attempts to maintain enough active (or available) threads by dynamically adding, suspending, or resuming internal worker threads, even if some tasks are stalled waiting to join others. However, no such adjustments are guaranteed in the face of blocked I/O or other unmanaged synchronization. The nested ForkJoinPool.ManagedBlocker interface enables extension of the kinds of synchronization accommodated. The default policies may be overridden using a constructor with parameters corresponding to those documented in class ThreadPoolExecutor.

In addition to execution and lifecycle control methods, this class provides status check methods (for example getStealCount()) that are intended to aid in developing, tuning, and monitoring fork/join applications. Also, method

`toString()` returns indications of pool state in a convenient form for informal monitoring.

As is the case with other ExecutorServices, there are three main task execution methods summarized in the following table. These are designed to be used primarily by clients not already engaged in fork/join computations in the current pool. The main forms of these methods accept instances of `ForkJoinTask`, but overloaded forms also allow mixed execution of plain `Runnable`- or `Callable`- based activities as well. However, tasks that are already executing in a pool should normally instead use the within-computation forms listed in the table unless using async event-style tasks that are not usually
joined, in which case there is little difference among choice of methods.

Summary of task execution methods

	Call from non-fork/join clients	**Call from within fork/join computations**
Arrange async execution	`execute(ForkJoinTask)`	`ForkJoinTask.fork()`
Await and obtain result	`invoke(ForkJoinTask)`	`ForkJoinTask.invoke()`
Arrange exec and obtain Future	`submit(ForkJoinTask)`	`ForkJoinTask.fork()` (ForkJoinTasks *are* Futures)

Additionally, this class supports `ScheduledExecutorService` methods to delay or periodically execute tasks, as well as method `submitWithTimeout(Callable, long, TimeUnit, Consumer)` to cancel tasks that take too long. The scheduled functions or actions may create and invoke other ForkJoinTasks. Delayed actions become enabled for execution and behave as ordinary submitted tasks when their delays elapse. Scheduling methods return ForkJoinTasks that implement the `ScheduledFuture` interface. Resource exhaustion encountered after initial submission results in task cancellation. When time-based methods are used, shutdown policies match the default policies of class `ScheduledThreadPoolExecutor`: upon `shutdown()`, existing periodic tasks will not re-execute, and the pool terminates when quiescent and existing delayed tasks complete. Method `cancelDelayedTasksOnShutdown()` may be used to disable all delayed tasks upon shutdown, and method `shutdownNow()` may be used to instead unconditionally initiate pool termination. Monitoring methods such as `getQueuedTaskCount()` do not include scheduled tasks that are not yet enabled for execution, which are reported separately by method `getDelayedTaskCount()`.

The parameters used to construct the common pool may be controlled by setting the following system properties:

- `java.util.concurrent.ForkJoinPool.common.parallelism` - the parallelism level, a non-negative integer. Usage is discouraged. Use `setParallelism(int)` instead.
- `java.util.concurrent.ForkJoinPool.common.threadFactory` - the class name of a `ForkJoinPool.ForkJoinWorkerThreadFactory`. The system class loader is used to load this class.
- `java.util.concurrent.ForkJoinPool.common.exceptionHandler` - the class name of a `Thread.UncaughtExceptionHandler`. The system class loader is used to load this class.
- `java.util.concurrent.ForkJoinPool.common.maximumSpares` - the maximum number of allowed extra threads to maintain target parallelism (default 256).

If no thread factory is supplied via a system property, then the common pool uses a factory that uses the system class loader as the thread context class loader. Upon any error in establishing these settings, default parameters are used. It is possible to disable use of threads by using a factory that may return `null`, in which case some tasks may never execute. While possible, it is strongly discouraged to set the parallelism property to zero, which may be internally overridden in the presence of intrinsically async tasks.

Implementation Note:

This implementation restricts the maximum number of running threads to 32767. Attempts to create pools with greater than the maximum number result in `IllegalArgumentException`. Also, this implementation rejects submitted tasks (that is, by throwing `RejectedExecutionException`) only when the pool is shut down or internal resources have been exhausted.

Since:

1.7

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
`static interface`	**`ForkJoinPool.ForkJoinWor`**	Factory for creating new `ForkJoinWorkerThread`s.
`static interface`	**`ForkJoinPool.ManagedBloc`**	Interface for extending managed parallelism for

	tasks running in ForkJoinPools.

Field Summary

Fields

Modifier and Type	Field	Description
static final ForkJoinPool.ForkJoi	**defaultForkJoinWorkerThr**	Creates a new ForkJoinWorkerThread.

Constructor Summary

Constructors

Constructor	Description
ForkJoinPool()	Creates a ForkJoinPool with parallelism equal to Runtime.availableProcessors(), using defaults for all other parameters (see ForkJoinPool(int, ForkJoinWorkerThreadFactory, UncaughtExceptionHandler, boolean, int, int, int, Predicate, long, TimeUnit)).
ForkJoinPool(int parallelism)	Creates a ForkJoinPool with the indicated parallelism level, using defaults for all other parameters (see ForkJoinPool(int, ForkJoinWorkerThreadFactory, UncaughtExceptionHandler, boolean, int, int, int, Predicate, long, TimeUnit)).
ForkJoinPool(int parallelism, ForkJoinPool.ForkJoinWorkerThreadFa Thread.UncaughtExceptionHandler ha boolean asyncMode)	Creates a ForkJoinPool with the given parameters (using defaults for others -- see ForkJoinPool(int, ForkJoinWorkerThreadFactory, UncaughtExceptionHandler,

	boolean, int, int, int, Predicate, long, TimeUnit)).
ForkJoinPool(int parallelism, ForkJoinPool.ForkJoinWorkerThreadFa Thread.UncaughtExceptionHandler ha boolean asyncMode, int corePoolSize, int maximumPoolSize, int minimumRunnable, Predicate<? super ForkJoinPool> saturate, long keepAliveTime, TimeUnit unit)	Creates a ForkJoinPool with the given parameters.

Method Summary

All Methods	Static Methods	

Modifier and Type	Method	Description
boolean	**awaitQuiescence**(long timeout, TimeUnit unit)	If called by a ForkJoinTask operating in this pool, equivalent in effect to ForkJoinTask.helpQuiesce().
boolean	**awaitTermination**(long timeout, TimeUnit unit)	Blocks until all tasks have completed execution after a shutdown request, or the timeout occurs, or the current thread is interrupted, whichever happens first.
void	**cancelDelayedTasksOnShut**	Arranges that scheduled tasks that are not executing and have not already been enabled for execution will not be

		executed and will be cancelled upon `shutdown()` (unless this pool is the `commonPool()` which never shuts down).
`void`	**`close`**`()`	Unless this is the `commonPool()`, initiates an orderly shutdown in which previously submitted tasks are executed, but no new tasks will be accepted, and waits until all tasks have completed execution and the executor has terminated.
`static ForkJoinPool`	**`commonPool`**`()`	Returns the common pool instance.
`protected int`	**`drainTasksTo`**`(Collection<? super ForkJoinTask<?>> c)`	Removes all available unexecuted submitted and forked tasks from scheduling queues and adds them to the given collection, without altering their execution status.
`void`	**`execute`**`(Runnable task)`	Executes the given command at some time in the future.
`void`	**`execute`**`(ForkJoinTask<?> task)`	Arranges for (asynchronous) execution of the given task.
`<T> ForkJoinTask<T>`		

	externalSubmit(ForkJoinTask<T> task)	Submits the given task as if submitted from a non-ForkJoinTask client.
int	**getActiveThreadCount**()	Returns an estimate of the number of threads that are currently stealing or executing tasks.
boolean	**getAsyncMode**()	Returns true if this pool uses local first-in-first-out scheduling mode for forked tasks that are never joined.
static int	**getCommonPoolParallelism**	Returns the targeted parallelism level of the common pool.
long	**getDelayedTaskCount**()	Returns an estimate of the number of delayed (including periodic) tasks scheduled in this pool that are not yet ready to submit for execution.
ForkJoinPool.ForkJoi	**getFactory**()	Returns the factory used for constructing new workers.
int	**getParallelism**()	Returns the targeted parallelism level of this pool.
int	**getPoolSize**()	Returns the number of worker threads that have started but not yet terminated.
int	**getQueuedSubmissionCount**	Returns an estimate of the number of tasks submitted to this pool that have not yet begun executing.

long	**getQueuedTaskCount**()	Returns an estimate of the total number of tasks currently held in queues by worker threads (but not including tasks submitted to the pool that have not begun executing).
int	**getRunningThreadCount**()	Returns an estimate of the number of worker threads that are not blocked waiting to join tasks or for other managed synchronization.
long	**getStealCount**()	Returns an estimate of the total number of completed tasks that were executed by a thread other than their submitter.
Thread.UncaughtExcep	**getUncaughtExceptionHand**	Returns the handler for internal worker threads that terminate due to unrecoverable errors encountered while executing tasks.
boolean	**hasQueuedSubmissions**()	Returns `true` if there are any tasks submitted to this pool that have not yet begun executing.
<T> T	**invoke**(ForkJoinTask<T> task)	Performs the given task, returning its result upon completion.
<T> List<Future<T>>	**invokeAllUninterruptibly**(Collection<? extends Callable<T>> tasks)	Uninterrupible version of `invokeAll`.
boolean	**isQuiescent**()	

		Returns `true` if all worker threads are currently idle.
boolean	**isShutdown**()	Returns `true` if this pool has been shut down.
boolean	**isTerminated**()	Returns `true` if all tasks have completed following shut down.
boolean	**isTerminating**()	Returns `true` if the process of termination has commenced but not yet completed.
<T> ForkJoinTask<T>	**lazySubmit**(ForkJoinTask<T> task)	Submits the given task without guaranteeing that it will eventually execute in the absence of available active threads.
static void	**managedBlock**(ForkJoinPool.ManagedBlo	Runs the given possibly blocking task.
protected ForkJoinTask<?>	**pollSubmission**()	Removes and returns the next unexecuted submission if one is available.
ScheduledFuture<?>	**schedule**(Runnable command, long delay, TimeUnit unit)	Submits a one-shot task that becomes enabled for execution after the given delay.
<V> ScheduledFuture<V>	**schedule**(Callable<V> callable, long delay, TimeUnit unit)	Submits a value-returning one-shot task that becomes enabled for execution after the given delay.
ScheduledFuture<?>	**scheduleAtFixedRate**(Runnable command, long initialDelay,	Submits a periodic action that becomes enabled for execution first after the given

	long period, TimeUnit unit)	initial delay, and subsequently with the given period; that is, executions will commence after initialDelay, then initialDelay + period, then initialDelay + 2 * period, and so on.
ScheduledFuture<?>	**scheduleWithFixedDelay** (Runnable command, long initialDelay, long delay, TimeUnit unit)	Submits a periodic action that becomes enabled for execution first after the given initial delay, and subsequently with the given delay between the termination of one execution and the commencement of the next.
int	**setParallelism** (int size)	Changes the target parallelism of this pool, controlling the future creation, use, and termination of worker threads.
void	**shutdown**()	Possibly initiates an orderly shutdown in which previously submitted tasks are executed, but no new tasks will be accepted.
List<Runnable>	**shutdownNow**()	Possibly attempts to cancel and/or stop all tasks, and reject all subsequently submitted tasks.

ForkJoinTask<?>	**submit**(Runnable task)	Submits a Runnable task for execution and returns a Future representing that task.
<T> ForkJoinTask<T>	**submit**(Runnable task, T result)	Submits a Runnable task for execution and returns a Future representing that task.
<T> ForkJoinTask<T>	**submit**(Callable<T> task)	Submits a value-returning task for execution and returns a Future representing the pending results of the task.
<T> ForkJoinTask<T>	**submit**(ForkJoinTask<T> task)	Submits a ForkJoinTask for execution.
<V> ForkJoinTask<V>	**submitWithTimeout**(Callable<V> callable, long timeout, TimeUnit unit, Consumer<? super ForkJoinTask<V>> timeout	Submits a task executing the given function, cancelling the task or performing a given timeoutAction if not completed within the given timeout period.
String	**toString**()	Returns a string identifying this pool, as well as its state, including indications of run state, parallelism level, and worker and task counts.

Methods declared in class AbstractExecutorService

invokeAll, invokeAll, invokeAny, invokeAny, newTaskFor, newTaskFor

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Field Details

defaultForkJoinWorkerThreadFactory

```
public static
final ForkJoinPool.ForkJoinWorkerThreadFactory defaultForkJoinWorkerThreadFactory
```

Creates a new ForkJoinWorkerThread. This factory is used unless overridden in ForkJoinPool constructors.

Constructor Details

ForkJoinPool

```
public ForkJoinPool()
```

Creates a `ForkJoinPool` with parallelism equal to `Runtime.availableProcessors()`, using defaults for all other parameters (see `ForkJoinPool(int, ForkJoinWorkerThreadFactory, UncaughtExceptionHandler, boolean, int, int, int, Predicate, long, TimeUnit)`).

ForkJoinPool

```
public ForkJoinPool(int parallelism)
```

Creates a `ForkJoinPool` with the indicated parallelism level, using defaults for all other parameters (see `ForkJoinPool(int, ForkJoinWorkerThreadFactory, UncaughtExceptionHandler, boolean, int, int, int, Predicate, long, TimeUnit)`).

Parameters:

`parallelism` - the parallelism level

Throws:

`IllegalArgumentException` - if parallelism less than or equal to zero, or greater than implementation limit

ForkJoinPool

```
public ForkJoinPool(int parallelism,
                    ForkJoinPool.ForkJoinWorkerThreadFactory factory,
                    Thread.UncaughtExceptionHandler handler,
                    boolean asyncMode)
```

Creates a `ForkJoinPool` with the given parameters (using defaults for others -- see `ForkJoinPool(int, ForkJoinWorkerThreadFactory, UncaughtExceptionHandler, boolean, int, int, int, Predicate, long, TimeUnit)`).

Parameters:

`parallelism` - the parallelism level. For default value, use `Runtime.availableProcessors()`.

`factory` - the factory for creating new threads. For default value, use `defaultForkJoinWorkerThreadFactory`.

`handler` - the handler for internal worker threads that terminate due to unrecoverable errors encountered while executing tasks. For default value, use `null`.

`asyncMode` - if true, establishes local first-in-first-out scheduling mode for forked tasks that are never joined. This mode may be more appropriate than default locally stack-based mode in applications in which worker threads only process event-style asynchronous tasks. For default value, use `false`.

Throws:

`IllegalArgumentException` - if parallelism less than or equal to zero, or greater than implementation limit

`NullPointerException` - if the factory is null

ForkJoinPool

```
public ForkJoinPool(int parallelism,
                    ForkJoinPool.ForkJoinWorkerThreadFactory factory,
                    Thread.UncaughtExceptionHandler handler,
                    boolean asyncMode,
                    int corePoolSize,
                    int maximumPoolSize,
                    int minimumRunnable,
                    Predicate<? super ForkJoinPool> saturate,
                    long keepAliveTime,
                    TimeUnit unit)
```

Creates a `ForkJoinPool` with the given parameters.

Parameters:

`parallelism` - the parallelism level. For default value, use `Runtime.availableProcessors()`.

`factory` - the factory for creating new threads. For default value, use `defaultForkJoinWorkerThreadFactory`.

`handler` - the handler for internal worker threads that terminate due to unrecoverable errors encountered while executing tasks. For default value, use `null`.

`asyncMode` - if true, establishes local first-in-first-out scheduling mode for forked tasks that are never joined. This mode may be more appropriate than default locally stack-based mode in applications in which worker threads only process event-style asynchronous tasks. For default value, use `false`.

`corePoolSize` - ignored: used in previous releases of this class but no longer applicable. Using `0` maintains compatibility across releases.

`maximumPoolSize` - the maximum number of threads allowed. When the maximum is reached, attempts to replace blocked threads fail. (However, because creation and termination of different threads may overlap, and may be managed by the given thread factory, this value may be transiently exceeded.) To arrange the same value as is used by default for the common pool, use `256` plus the `parallelism` level. (By default, the common pool allows a maximum of 256 spare threads.) Using a value (for example `Integer.MAX_VALUE`) larger than the implementation's total thread limit has the same effect as using this limit (which is the default).

`minimumRunnable` - the minimum allowed number of core threads not blocked by a join or `ForkJoinPool.ManagedBlocker`. To ensure progress, when too few unblocked threads exist and unexecuted tasks may exist, new threads are constructed, up to the given maximumPoolSize. For the default value, use `1`, that ensures liveness. A larger value might improve throughput in the presence of blocked activities, but might not, due to increased overhead. A value of zero may be acceptable when submitted tasks cannot have dependencies requiring additional threads.

`saturate` - if non-null, a predicate invoked upon attempts to create more than the maximum total allowed threads. By default, when a thread is about to block on a join or `ForkJoinPool.ManagedBlocker`, but cannot be replaced because the maximumPoolSize would be exceeded, a `RejectedExecutionException` is thrown. But if this predicate returns `true`, then no exception is thrown, so the pool continues to operate with fewer than the target number of runnable threads, which might not ensure progress.

`keepAliveTime` - the elapsed time since last use before a thread is terminated (and then later replaced if needed). For the default value, use `60, TimeUnit.SECONDS`.

`unit` - the time unit for the `keepAliveTime` argument

Throws:

`IllegalArgumentException` - if parallelism is less than or equal to zero, or is greater than implementation limit, or if maximumPoolSize is less than parallelism, of if the keepAliveTime is less than or equal to zero.

`NullPointerException` - if the factory is null

Since:

9

Method Details

commonPool

```
public static ForkJoinPool commonPool()
```

Returns the common pool instance. This pool is statically constructed; its run state is unaffected by attempts to `shutdown()` or `shutdownNow()`. However this pool and any ongoing processing are automatically terminated upon program `System.exit(int)`. Any program that relies on asynchronous task processing to complete before program termination should invoke `commonPool().awaitQuiescence`, before exit.

Returns:

the common pool instance

Since:

1.8

invoke

```
public <T> T invoke(ForkJoinTask<T> task)
```

Performs the given task, returning its result upon completion. If the computation encounters an unchecked Exception or Error, it is rethrown as the outcome of this invocation. Rethrown exceptions behave in the same way as regular exceptions, but, when possible, contain stack traces (as displayed for example using `ex.printStackTrace()`) of both the current thread as well as the thread actually encountering the exception; minimally only the latter.

Type Parameters:

`T` - the type of the task's result

Parameters:

`task` - the task

Returns:

the task's result

Throws:

`NullPointerException` - if the task is null

`RejectedExecutionException` - if the task cannot be scheduled for execution

execute

```
public void execute(ForkJoinTask<?> task)
```

Arranges for (asynchronous) execution of the given task.

Parameters:

`task` - the task

Throws:

`NullPointerException` - if the task is null

`RejectedExecutionException` - if the task cannot be scheduled for execution

execute

```
public void execute(Runnable task)
```

Description copied from interface: `Executor`

Executes the given command at some time in the future. The command may execute in a new thread, in a pooled thread, or in the calling thread, at the discretion of the `Executor` implementation.

Specified by:

`execute` in interface `Executor`

Parameters:

`task` - the runnable task

Throws:

`NullPointerException` - if the task is null

`RejectedExecutionException` - if the task cannot be scheduled for execution

submit

```
public <T> ForkJoinTask<T> submit(ForkJoinTask<T> task)
```

Submits a ForkJoinTask for execution.

Implementation Requirements:

This method is equivalent to `externalSubmit(ForkJoinTask)` when called from a thread that is not in this pool.

Type Parameters:

T - the type of the task's result

Parameters:

`task` - the task to submit

Returns:

the task

Throws:

`NullPointerException` - if the task is null

`RejectedExecutionException` - if the task cannot be scheduled for execution

submit

```
public <T> ForkJoinTask<T> submit(Callable<T> task)
```

Description copied from interface: `ExecutorService`

Submits a value-returning task for execution and returns a Future representing the pending results of the task. The Future's `get` method will return the task's result upon successful completion.

If you would like to immediately block waiting for a task, you can use constructions of the form `result = exec.submit(aCallable).get();`

Note: The `Executors` class includes a set of methods that can convert some other common closure-like objects, for example, `PrivilegedAction` to `Callable` form so they can be submitted.

Specified by:

`submit` in interface `ExecutorService`

Overrides:

`submit` in class `AbstractExecutorService`

Type Parameters:

T - the type of the task's result

Parameters:

`task` - the task to submit

Returns:

a Future representing pending completion of the task

Throws:

`NullPointerException` - if the task is null

RejectedExecutionException - if the task cannot be scheduled for execution

submit

```
public <T> ForkJoinTask<T> submit(Runnable task,
                                  T result)
```

Description copied from interface: ExecutorService

Submits a Runnable task for execution and returns a Future representing that task. The Future's get method will return the given result upon successful completion.

Specified by:

submit in interface ExecutorService

Overrides:

submit in class AbstractExecutorService

Type Parameters:

T - the type of the result

Parameters:

task - the task to submit

result - the result to return

Returns:

a Future representing pending completion of the task

Throws:

NullPointerException - if the task is null

RejectedExecutionException - if the task cannot be scheduled for execution

submit

```
public ForkJoinTask<?> submit(Runnable task)
```

Description copied from interface: ExecutorService

Submits a Runnable task for execution and returns a Future representing that task. The Future's get method will return null upon *successful* completion.

Specified by:

submit in interface ExecutorService

Overrides:

submit in class AbstractExecutorService

Parameters:

`task` - the task to submit

Returns:

a Future representing pending completion of the task

Throws:

`NullPointerException` - if the task is null

`RejectedExecutionException` - if the task cannot be scheduled for execution

externalSubmit

```
public <T> ForkJoinTask<T> externalSubmit(ForkJoinTask<T> task)
```

Submits the given task as if submitted from a non-`ForkJoinTask` client. The task is added to a scheduling queue for submissions to the pool even when called from a thread in the pool.

Implementation Requirements:

This method is equivalent to `submit(ForkJoinTask)` when called from a thread that is not in this pool.

Type Parameters:

`T` - the type of the task's result

Parameters:

`task` - the task to submit

Returns:

the task

Throws:

`NullPointerException` - if the task is null

`RejectedExecutionException` - if the task cannot be scheduled for execution

Since:

20

lazySubmit

```
public <T> ForkJoinTask<T> lazySubmit(ForkJoinTask<T> task)
```

Submits the given task without guaranteeing that it will eventually execute in the absence of available active threads. In some contexts, this method may reduce contention and overhead by relying on context-specific knowledge that existing threads (possibly including the calling thread if operating in this pool) will eventually be available to execute the task.

Type Parameters:

T - the type of the task's result

Parameters:

`task` - the task

Returns:

the task

Throws:

`NullPointerException` - if the task is null

`RejectedExecutionException` - if the task cannot be scheduled for execution

Since:

19

setParallelism

```
public int setParallelism(int size)
```

Changes the target parallelism of this pool, controlling the future creation, use, and termination of worker threads. Applications include contexts in which the number of available processors changes over time.

Implementation Note:

This implementation restricts the maximum number of running threads to 32767

Parameters:

`size` - the target parallelism level

Returns:

the previous parallelism level.

Throws:

`IllegalArgumentException` - if size is less than 1 or greater than the maximum supported by this pool.

`UnsupportedOperationException` - this is the`commonPool()` and parallelism level was set by System property `java.util.concurrent.ForkJoinPool.common.parallelism`.

Since:

19

invokeAllUninterruptibly

```
public <T> List<Future<T>> invokeAllUninterruptibly
(Collection<? extends Callable<T>> tasks)
```

Uninterrupible version of `invokeAll`. Executes the given tasks, returning a list of Futures holding their status and results when all complete, ignoring interrupts. `Future.isDone()` is `true` for each element of the returned list. Note that a *completed* task could have terminated either normally or by throwing an exception. The results of this method are undefined if the given collection is modified while this operation is in progress.

API Note:

This method supports usages that previously relied on an incompatible override of `ExecutorService.invokeAll(java.util.Collection)`.

Type Parameters:

`T` - the type of the values returned from the tasks

Parameters:

`tasks` - the collection of tasks

Returns:

a list of Futures representing the tasks, in the same sequential order as produced by the iterator for the given task list, each of which has completed

Throws:

`NullPointerException` - if tasks or any of its elements are `null`

`RejectedExecutionException` - if any task cannot be scheduled for execution

Since:

22

schedule

```
public ScheduledFuture<?> schedule(Runnable command,
                                   long delay,
                                   TimeUnit unit)
```

Submits a one-shot task that becomes enabled for execution after the given delay. At that point it will execute unless explicitly cancelled, or fail to execute (eventually reporting cancellation) when encountering resource exhaustion, or the pool is `shutdownNow()`, or is `shutdown()` when otherwise quiescent and `cancelDelayedTasksOnShutdown()` is in effect.

Specified by:

`schedule` in interface `ScheduledExecutorService`

Parameters:

`command` - the task to execute

`delay` - the time from now to delay execution

`unit` - the time unit of the delay parameter

Returns:

a ForkJoinTask implementing the ScheduledFuture interface, whose `get()` method will return `null` upon normal completion.

Throws:

`RejectedExecutionException` - if the pool is shutdown or submission encounters resource exhaustion.

`NullPointerException` - if command or unit is null

Since:

25

schedule

```
public <V> ScheduledFuture<V> schedule(Callable<V> callable,
                                       long delay,
                                       TimeUnit unit)
```

Submits a value-returning one-shot task that becomes enabled for execution after the given delay. At that point it will execute unless explicitly cancelled, or fail to execute (eventually reporting cancellation) when encountering resource exhaustion, or the pool is `shutdownNow()`, or is `shutdown()` when otherwise quiescent and `cancelDelayedTasksOnShutdown()` is in effect.

Specified by:

`schedule` in interface `ScheduledExecutorService`

Type Parameters:

`V` - the type of the callable's result

Parameters:

`callable` - the function to execute

`delay` - the time from now to delay execution

`unit` - the time unit of the delay parameter

Returns:

a ForkJoinTask implementing the ScheduledFuture interface, whose `get()` method will return the value from the callable upon normal completion.

Throws:

`RejectedExecutionException` - if the pool is shutdown or submission encounters resource exhaustion.

`NullPointerException` - if command or unit is null

Since:

25

scheduleAtFixedRate

```
public ScheduledFuture<?> scheduleAtFixedRate(Runnable command,
                                              long initialDelay,
                                              long period,
                                              TimeUnit unit)
```

Submits a periodic action that becomes enabled for execution first after the given initial delay, and subsequently with the given period; that is, executions will commence after `initialDelay`, then `initialDelay + period`, then `initialDelay + 2 * period`, and so on.

The sequence of task executions continues indefinitely until one of the following exceptional completions occur:

- The task is explicitly cancelled
- Method `shutdownNow()` is called
- Method `shutdown()` is called and the pool is otherwise quiescent, in which case existing executions continue but subsequent executions do not.
- An execution or the task encounters resource exhaustion.
- An execution of the task throws an exception. In this case calling `get` on the returned future will throw `ExecutionException`, holding the exception as its cause.

Subsequent executions are suppressed. Subsequent calls to `isDone()` on the returned future will return `true`.

If any execution of this task takes longer than its period, then subsequent executions may start late, but will not concurrently execute.

Specified by:

`scheduleAtFixedRate` in interface `ScheduledExecutorService`

Parameters:

`command` - the task to execute

`initialDelay` - the time to delay first execution

`period` - the period between successive executions

`unit` - the time unit of the initialDelay and period parameters

Returns:

a ForkJoinTask implementing the ScheduledFuture interface. The future's `get()` method will never return normally, and will throw an exception upon task cancellation or abnormal termination of a task execution.

Throws:

`RejectedExecutionException` - if the pool is shutdown or submission encounters resource exhaustion.

`NullPointerException` - if command or unit is null

`IllegalArgumentException` - if period less than or equal to zero

Since:

25

scheduleWithFixedDelay

```
public ScheduledFuture<?> scheduleWithFixedDelay(Runnable command,
                                                 long initialDelay,
                                                 long delay,
                                                 TimeUnit unit)
```

Submits a periodic action that becomes enabled for execution first after the given initial delay, and subsequently with the given delay between the termination of one execution and the commencement of the next.

The sequence of task executions continues indefinitely until one of the following exceptional completions occur:

- The task is explicitly cancelled
- Method `shutdownNow()` is called
- Method `shutdown()` is called and the pool is otherwise quiescent, in which case existing executions continue but subsequent executions do not.
- An execution or the task encounters resource exhaustion.
- An execution of the task throws an exception. In this case calling `get` on the returned future will throw `ExecutionException`, holding the exception as its cause.

Subsequent executions are suppressed. Subsequent calls to `isDone()` on the returned future will return `true`.

Specified by:

`scheduleWithFixedDelay` in interface `ScheduledExecutorService`

Parameters:

`command` - the task to execute

`initialDelay` - the time to delay first execution

`delay` - the delay between the termination of one execution and the commencement of the next

`unit` - the time unit of the initialDelay and delay parameters

Returns:

a ForkJoinTask implementing the ScheduledFuture interface. The future's `get()` method will never return normally, and will throw an exception upon task cancellation or abnormal termination of a task execution.

Throws:

`RejectedExecutionException` - if the pool is shutdown or submission encounters resource exhaustion.

`NullPointerException` - if command or unit is null

`IllegalArgumentException` - if delay less than or equal to zero

Since:

25

submitWithTimeout

```
public <V> ForkJoinTask<V> submitWithTimeout
(Callable<V> callable,
 long timeout,
 TimeUnit unit,
 Consumer<? super ForkJoinTask<V>> timeoutAction)
```

Submits a task executing the given function, cancelling the task or performing a given timeoutAction if not completed within the given timeout period. If the optional `timeoutAction` is null, the task is cancelled (via `cancel(true)`. Otherwise, the action is applied and the task may be interrupted if running. Actions may include `ForkJoinTask.complete(V)` to set a replacement value or `ForkJoinTask.completeExceptionally(Throwable)` to throw an appropriate exception. Note that these can succeed only if the task has not already completed when the timeoutAction executes.

Type Parameters:

`V` - the type of the callable's result

Parameters:

`callable` - the function to execute

`timeout` - the time to wait before cancelling if not completed

`unit` - the time unit of the timeout parameter

`timeoutAction` - if nonnull, an action to perform on timeout, otherwise the default action is to cancel using `cancel(true)`.

Returns:

a Future that can be used to extract result or cancel

Throws:

`RejectedExecutionException` - if the task cannot be scheduled for execution

NullPointerException - if callable or unit is null

Since:

25

cancelDelayedTasksOnShutdown

```
public void cancelDelayedTasksOnShutdown()
```

Arranges that scheduled tasks that are not executing and have not already been enabled for execution will not be executed and will be cancelled upon shutdown() (unless this pool is the commonPool() which never shuts down). This method may be invoked either before shutdown() to take effect upon the next call, or afterwards to cancel such tasks, which may then allow termination. Note that subsequent executions of periodic tasks are always disabled upon shutdown, so this method applies meaningfully only to non-periodic tasks.

Since:

25

getFactory

```
public ForkJoinPool.ForkJoinWorkerThreadFactory getFactory()
```

Returns the factory used for constructing new workers.

Returns:

the factory used for constructing new workers

getUncaughtExceptionHandler

```
public Thread.UncaughtExceptionHandler getUncaughtExceptionHandler()
```

Returns the handler for internal worker threads that terminate due to unrecoverable errors encountered while executing tasks.

Returns:

the handler, or `null` if none

getParallelism

```
public int getParallelism()
```

Returns the targeted parallelism level of this pool.

Returns:

the targeted parallelism level of this pool

getCommonPoolParallelism

```
public static int getCommonPoolParallelism()
```

Returns the targeted parallelism level of the common pool.

Returns:

the targeted parallelism level of the common pool

Since:

1.8

getPoolSize

```
public int getPoolSize()
```

Returns the number of worker threads that have started but not yet terminated. The result returned by this method may differ from `getParallelism()` when threads are created to maintain parallelism when others are cooperatively blocked.

Returns:

the number of worker threads

getAsyncMode

```
public boolean getAsyncMode()
```

Returns `true` if this pool uses local first-in-first-out scheduling mode for forked tasks that are never joined.

Returns:

`true` if this pool uses async mode

getRunningThreadCount

```
public int getRunningThreadCount()
```

Returns an estimate of the number of worker threads that are not blocked waiting to join tasks or for other managed synchronization. This method may overestimate the number of running threads.

Returns:

the number of worker threads

getActiveThreadCount

```
public int getActiveThreadCount()
```

Returns an estimate of the number of threads that are currently stealing or executing tasks. This method may overestimate the number of active threads.

Returns:

the number of active threads

isQuiescent

```
public boolean isQuiescent()
```

Returns `true` if all worker threads are currently idle. An idle worker is one that cannot obtain a task to execute because none are available to steal from other threads, and there are no pending submissions to the pool. This method is conservative; it might not return `true` immediately upon idleness of all threads, but will eventually become true if threads remain inactive.

Returns:

`true` if all threads are currently idle

getStealCount

```
public long getStealCount()
```

Returns an estimate of the total number of completed tasks that were executed by a thread other than their submitter. The reported value underestimates the actual total number of steals when the pool is not quiescent. This value may be useful for monitoring and tuning fork/join programs: in general, steal counts should be high enough to keep threads busy, but low enough to avoid overhead and contention across threads.

Returns:

the number of steals

getQueuedTaskCount

```
public long getQueuedTaskCount()
```

Returns an estimate of the total number of tasks currently held in queues by worker threads (but not including tasks submitted to the pool that have not begun executing). This value is only an approximation, obtained by iterating across all threads in the pool. This method may be useful for tuning task granularities.The returned count does not include scheduled tasks that are not yet ready to execute, which are reported separately by method `getDelayedTaskCount()`.

Returns:

the number of queued tasks

See Also:

ForkJoinWorkerThread.getQueuedTaskCount()

getQueuedSubmissionCount

```
public int getQueuedSubmissionCount()
```

Returns an estimate of the number of tasks submitted to this pool that have not yet begun executing. This method may take time proportional to the number of submissions.

Returns:

the number of queued submissions

getDelayedTaskCount

```
public long getDelayedTaskCount()
```

Returns an estimate of the number of delayed (including periodic) tasks scheduled in this pool that are not yet ready to submit for execution. The returned value is inaccurate while delayed tasks are being processed.

Returns:

an estimate of the number of delayed tasks

Since:

25

hasQueuedSubmissions

```
public boolean hasQueuedSubmissions()
```

Returns `true` if there are any tasks submitted to this pool that have not yet begun executing.

Returns:

`true` if there are any queued submissions

pollSubmission

```
protected ForkJoinTask<?> pollSubmission()
```

Removes and returns the next unexecuted submission if one is available. This method may be useful in extensions to this class that re-assign work in systems with multiple pools.

Returns:

the next submission, or `null` if none

drainTasksTo

```
protected int drainTasksTo(Collection<? super ForkJoinTask<?>> c)
```

Removes all available unexecuted submitted and forked tasks from scheduling queues and adds them to the given collection, without altering their execution status. These may include artificially generated or wrapped tasks. This method is designed to be invoked only when the pool is known to be quiescent. Invocations at other times may not remove all tasks. A failure encountered while attempting to add elements to collection c may result in elements being in neither, either or both collections when the associated exception is thrown. The behavior of this operation is undefined if the specified collection is modified while the operation is in progress.

Parameters:

c - the collection to transfer elements into

Returns:

the number of elements transferred

toString

```
public String toString()
```

Returns a string identifying this pool, as well as its state, including indications of run state, parallelism level, and worker and task counts.

Overrides:

toString in class Object

Returns:

a string identifying this pool, as well as its state

shutdown

```
public void shutdown()
```

Possibly initiates an orderly shutdown in which previously submitted tasks are executed, but no new tasks will be accepted. Invocation has no effect on execution state if this is the commonPool(), and no additional effect if already shut down. Tasks that are in the process of being submitted concurrently during the course of this method may or may not be rejected.

Specified by:

shutdown in interface ExecutorService

shutdownNow

```
public List<Runnable> shutdownNow()
```

Possibly attempts to cancel and/or stop all tasks, and reject all subsequently submitted tasks. Invocation has no effect on execution state if this is the `commonPool()`, and no additional effect if already shut down. Otherwise, tasks that are in the process of being submitted or executed concurrently during the course of this method may or may not be rejected. This method cancels both existing and unexecuted tasks, in order to permit termination in the presence of task dependencies. So the method always returns an empty list (unlike the case for some other Executors).

Specified by:

`shutdownNow` in interface `ExecutorService`

Returns:

an empty list

isTerminated

```
public boolean isTerminated()
```

Returns `true` if all tasks have completed following shut down.

Specified by:

`isTerminated` in interface `ExecutorService`

Returns:

`true` if all tasks have completed following shut down

isTerminating

```
public boolean isTerminating()
```

Returns `true` if the process of termination has commenced but not yet completed. This method may be useful for debugging. A return of `true` reported a sufficient period after shutdown may indicate that submitted tasks have ignored or suppressed interruption, or are waiting for I/O, causing this executor not to properly terminate. (See the advisory notes for class `ForkJoinTask` stating that tasks should not normally entail blocking operations. But if they do, they must abort them on interrupt.)

Returns:

`true` if terminating but not yet terminated

isShutdown

```
public boolean isShutdown()
```

Returns `true` if this pool has been shut down.

Specified by:

isShutdown in interface ExecutorService

Returns:

true if this pool has been shut down

awaitTermination

```
public boolean awaitTermination(long timeout,
                                TimeUnit unit)
                         throws InterruptedException
```

Blocks until all tasks have completed execution after a shutdown request, or the timeout occurs, or the current thread is interrupted, whichever happens first. Because the commonPool() never terminates until program shutdown, when applied to the common pool, this method is equivalent to awaitQuiescence(long, TimeUnit) but always returns false.

Specified by:

awaitTermination in interface ExecutorService

Parameters:

timeout - the maximum time to wait

unit - the time unit of the timeout argument

Returns:

true if this executor terminated and false if the timeout elapsed before termination

Throws:

InterruptedException - if interrupted while waiting

awaitQuiescence

```
public boolean awaitQuiescence(long timeout,
                               TimeUnit unit)
```

If called by a ForkJoinTask operating in this pool, equivalent in effect to ForkJoinTask.helpQuiesce(). Otherwise, waits and/or attempts to assist performing tasks until this pool isQuiescent() or the indicated timeout elapses.

Parameters:

timeout - the maximum time to wait

unit - the time unit of the timeout argument

Returns:

true if quiescent; false if the timeout elapsed.

close

```
public void close()
```

Unless this is the `commonPool()`, initiates an orderly shutdown in which previously submitted tasks are executed, but no new tasks will be accepted, and waits until all tasks have completed execution and the executor has terminated.

If already terminated, or this is the `commonPool()`, this method has no effect on execution, and does not wait. Otherwise, if interrupted while waiting, this method stops all executing tasks as if by invoking `shutdownNow()`. It then continues to wait until all actively executing tasks have completed. Tasks that were awaiting execution are not executed. The interrupted status will be re-asserted before this method returns.

Specified by:

`close` in interface `AutoCloseable`

Specified by:

`close` in interface `ExecutorService`

Since:

19

managedBlock

```
public static void managedBlock(ForkJoinPool.ManagedBlocker blocker)
                         throws InterruptedException
```

Runs the given possibly blocking task. When running in a ForkJoinPool, this method possibly arranges for a spare thread to be activated if necessary to ensure sufficient parallelism while the current thread is blocked in `blocker.block()`.

This method repeatedly calls `blocker.isReleasable()` and `blocker.block()` until either method returns `true`. Every call to `blocker.block()` is preceded by a call to `blocker.isReleasable()` that returned `false`.

If not running in a ForkJoinPool, this method is behaviorally equivalent to

```
while (!blocker.isReleasable())
  if (blocker.block())
    break;
```

If running in a ForkJoinPool, the pool may first be expanded to ensure sufficient parallelism available during the call to `blocker.block()`.

Parameters:

`blocker` - the blocker task

Throws:

`InterruptedException` - if `blocker.block()` did so

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.